BA305 — Exam 1 · Version 2 · Solutions

Total: 66 points. Part A: 41 points · Part B: 25 points.

Part A — Concepts (Questions 1–15, 41 points)

No code in this part.

Part A: 41 points.

Question 1 — True / False (2 points)

The number of data points in a dataset is called its dimensionality.

- True
- False

> **Answer — False.** The number of rows is the data's size; the number of features (columns) is its dimensionality.

Question 2 — Multiple choice (2 points)

An online store's data has a column `Region` that holds a number from 1 to 9 for each customer's sales region. You plan to use it in a k-NN model. What should you do?

- (a) Keep it as is; it is already numeric.
- (b) Standardize it so it has the same scale as the other features.
- (c) Turn it into dummy (0/1) columns.
- (d) Divide it by 9 so it lies between 0 and 1.

> **Answer — (c).** Region numbers are names, not quantities: as numbers, k-NN would treat Region 9 as far from Region 1. Dummies make every pair of regions equally far apart. (b) and (d) only rescale the fake distance.

Question 3 — Multiple choice (2 points)

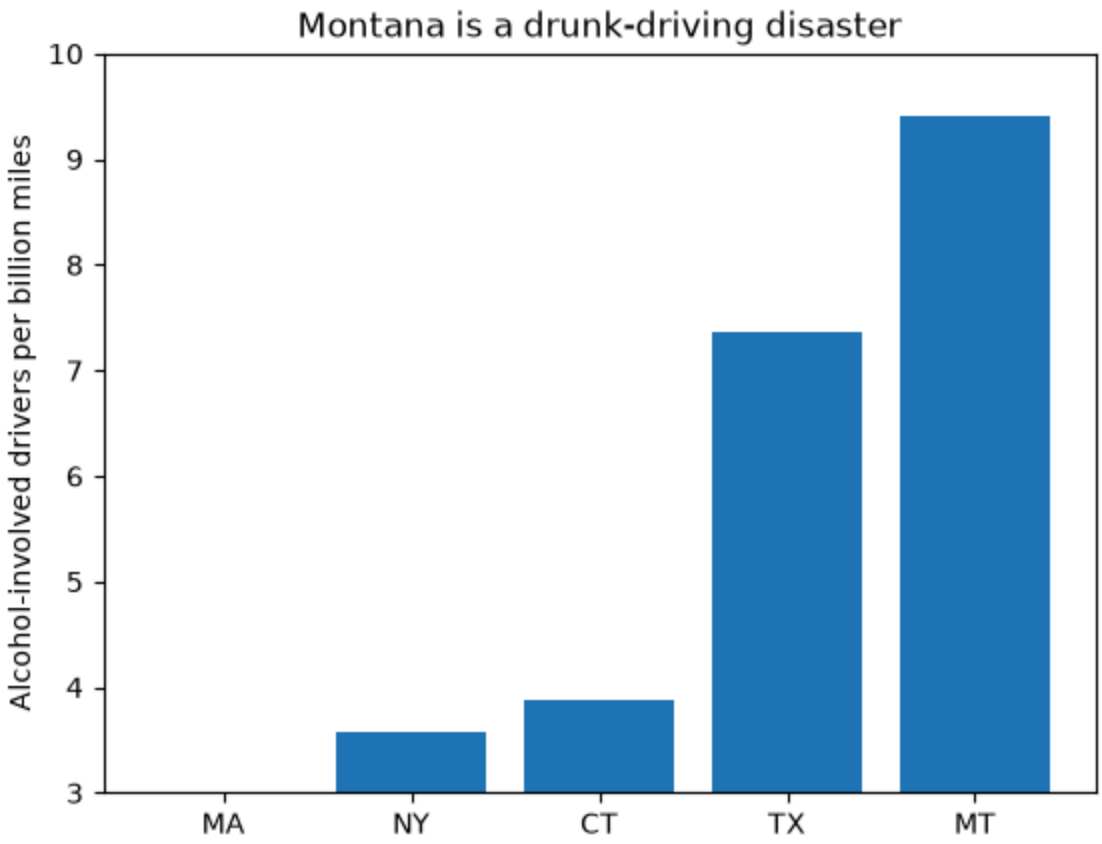
Customer ages have a mean of 50 and a standard deviation of 10. After standardizing age, what value does a 35-year-old customer get?

- (a) −1.5
- (b) 1.5
- (c) −15
- (d) 3.5

Answer — (a). z = (35 − 50) / 10 = −1.5.

Question 4 — True / False (2 points)

The chart below shows drivers in fatal crashes involving alcohol, per billion miles driven, in five US states.



True or false: according to this chart, Massachusetts (MA) had no alcohol-involved fatal crashes.

- True
- False

Answer — False. MA's rate is 2.87, below the axis start of 3, so its bar is cut off. A truncated axis hides and distorts bar lengths.

Question 5 — True / False (3 points)

On a scatterplot of countries, you color each point by its **GDP per person**, a continuous number. The best palette for this is a set of clearly different colors (red, green, blue, orange, purple), one per range of GDP.

- True
- False

> **Answer — False.** A continuous variable needs a sequential palette (light to dark). Distinct hues suit categories such as continent.

Question 6 — Multiple choice (3 points)

An analyst standardizes seven measures of US car crashes and car insurance and runs PCA. The data has one row per US state. Five columns are rates of drivers in fatal crashes per billion miles driven: `total`, and those who were `speeding`, `alcohol`-involved, `not_distracted`, or had `no_previous` accidents. `ins_premium` is the average yearly car-insurance premium ($) and `ins_losses` the insurers' yearly losses per insured driver ($). The table shows the correlation of each original column with the first two principal components.

	PC1	PC2
total	0.96	0.09
speeding	0.75	0.10
alcohol	0.91	0.04
not_distracted	0.88	0.05
no_previous	0.92	0.12
ins_premium	-0.26	0.86
ins_losses	-0.14	0.90

Which description fits best?

- (a) PC1 measures how dangerous a state's roads are; PC2 measures how much money goes through car insurance.
- (b) PC1 measures speeding only; PC2 measures alcohol only.
- (c) PC1 measures how much money goes through car insurance; PC2 measures crash rates.
- (d) The components cannot be interpreted because some correlations are negative.

> **Answer — (a).** All five crash columns load strongly on PC1 (0.75–0.96), and the two insurance columns load on PC2 (0.86, 0.90). (c) is reversed. (d) Negative correlations can be interpreted too.

Question 7 — Multiple choice (3 points)

A table of US car-crash statistics has 51 rows (one per state), 7 numeric columns and 1 text column (the state's two-letter code). After removing the text column and standardizing, what is the **largest** number of principal components PCA can return?

- (a) 7
- (b) 8
- (c) 51

- (d) 58

> **Answer — (a).** PCA cannot return more components than there are numeric columns: 7.

Question 8 — Multiple choice (2 points)

Which of the following is **always** true of the principal components of a dataset?

- (a) Each component is one of the original columns.
- (b) The components are uncorrelated with each other, and PC1 explains the most variance.
- (c) Every component explains the same share of the variance.
- (d) The first component is the original column with the largest variance.

> **Answer — (b).** Components are uncorrelated and ordered by variance. Each is a mix of columns, not a single column.

Question 9 — True / False (3 points)

In a card-transaction dataset, 95% of transactions are legitimate and 5% are fraud. A k-NN model reaches 94% accuracy on the test set. True or false: this model does worse than simply predicting "legitimate" for every transaction.

- True
- False

> **Answer — True.** Always predicting "legitimate" already scores 95%.

Question 10 — Multiple choice (3 points)

An email filter moves suspected spam into a junk folder that no one ever checks. Missing a real client email costs the firm far more than letting a spam email through. Which metric should the filter be tuned for, with spam as the positive class?

- (a) Recall
- (b) Precision
- (c) Accuracy
- (d) R^2

> **Answer — (b) Precision.** The costly error is a false positive (a real email lost), and precision is high when false positives are rare.

Question 11 — Select all that apply (3 points)

You run 5-fold cross-validation on 1,000 rows. Select **all** that are true.

- (a) The model is trained 5 times.
- (b) Each training run uses 800 rows.
- (c) Every row is in the test fold exactly once.
- (d) Every row is used for training exactly once.
- (e) The reported score is the average of the 5 test scores.

> **Answer — (a), (b), (c) and (e).** Five rounds, each trained on 800 rows and tested on 200. (d) Each row is used for training in 4 rounds, not once.

Question 12 — Select all that apply (4 points)

Select **all** that are true.

- (a) A model with AUC = 0.5 does no better than random guessing.
- (b) A model with AUC = 0.3 predicts every example incorrectly.
- (c) The ROC curve shows the true positive rate against the false positive rate over a range of thresholds.
- (d) An AUC of 0.9 means the model is right 90% of the time.

> **Answer — (a) and (c).** (b) AUC = 0.3 is worse than random but not all wrong (that would be AUC = 0). (d) AUC is not accuracy.

Question 13 — Multiple choice (3 points)

For a loan screening model, a missed defaulter (FN) costs $500 and wrongly rejecting a good applicant (FP) costs $50. On the test set:

Threshold	FN	FP
A	10	100
B	30	20

Which threshold costs less, and by how much?

- (a) A, by $6,000
- (b) B, by $6,000
- (c) B, because it makes fewer errors in total
- (d) They cost the same.

> **Answer — (a).** A costs $10,000 and B costs $16,000. A makes more errors, but cheaper ones.

Question 14 — Select all that apply (3 points)

A team uses k-NN on customer data, after properly scaling the features. Select **all** that are true.

- (a) Measuring income in thousands of dollars instead of dollars changes which customers are a record's nearest neighbors.
- (b) With k = 1, each prediction is based on a single training example.
- (c) A very large k (e.g., k = number of training rows) predicts the most common class for every customer.
- (d) Once trained, k-NN can discard the training data and keep only a formula.

> **Answer — (b) and (c).** (b) k = 1 copies a single neighbor. (c) k = all rows always predicts the most common class. (a) After standard scaling, the units no longer matter. (d) k-NN keeps the training data; the data is the model.

Question 15 — Multiple choice (3 points)

A bank wants to predict whether a customer will accept a loan offer. It splits its data into training and test sets and tries k-NN with several values of k. The table shows the accuracy of each model on both sets. What is the best choice of k?

	k	train accuracy	test accuracy
0	1	1.000	0.954
1	3	0.977	0.954
2	5	0.972	0.956
3	7	0.969	0.952
4	9	0.965	0.948
5	11	0.960	0.946
6	13	0.958	0.945
7	15	0.955	0.944
8	17	0.952	0.942
9	19	0.951	0.940

- (a) 1
- (b) 3
- (c) 5
- (d) 13
- (e) 19

> **Answer — (c) 5.** It has the highest test accuracy (0.956). k = 1's perfect training accuracy signals overfitting.

Part B — Coding questions (Questions 16–23, 25 points)

The code and its output are in two separate handouts, **Handout B1** and **Handout B2**. Each question names the exhibits it uses. You cannot run the code.

Part B: 25 points. Total: 66 points.

Case B1 — Titanic (use Handout B1)

891 passengers of the Titanic. Goal: predict `survived`. The columns:

Column	Meaning
`survived`	1 if the passenger survived, 0 if not (the target)
`pclass`	Ticket class as a number: 1 = first, 2 = second, 3 = third
`sex`	`male` or `female`
`age`	Age in years
`sibsp`	Number of siblings and spouses aboard
`parch`	Number of parents and children aboard
`fare`	Ticket price, in pounds
`embarked`	Port where the passenger boarded: S, C or Q
`class`	Ticket class as text: First, Second, Third
`who`	`man`, `woman` or `child`
`adult_male`	True if the passenger is an adult man
`deck`	Deck of the cabin, A to G
`embark_town`	Port where the passenger boarded, as a name: Southampton, Cherbourg, Queenstown
`alive`	`yes` if the passenger survived, `no` if not
`alone`	True if the passenger had no family aboard

Exhibit 1

```
# Load the passenger list.
titanic = pd.read_csv(url)
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

---

Question 16 — Multiple choice (3 points)

Exhibit 2

```
# Column types and non-missing counts.
titanic.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    str
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    str
 8   class        891 non-null    str
 9   who          891 non-null    str
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    str
 12  embark_town  889 non-null    str
 13  alive        891 non-null    str
 14  alone        891 non-null    bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 116.9 KB
```

`survived` is the target. Based only on this output, how many columns are available as features, and how many passengers have no missing value in any column?

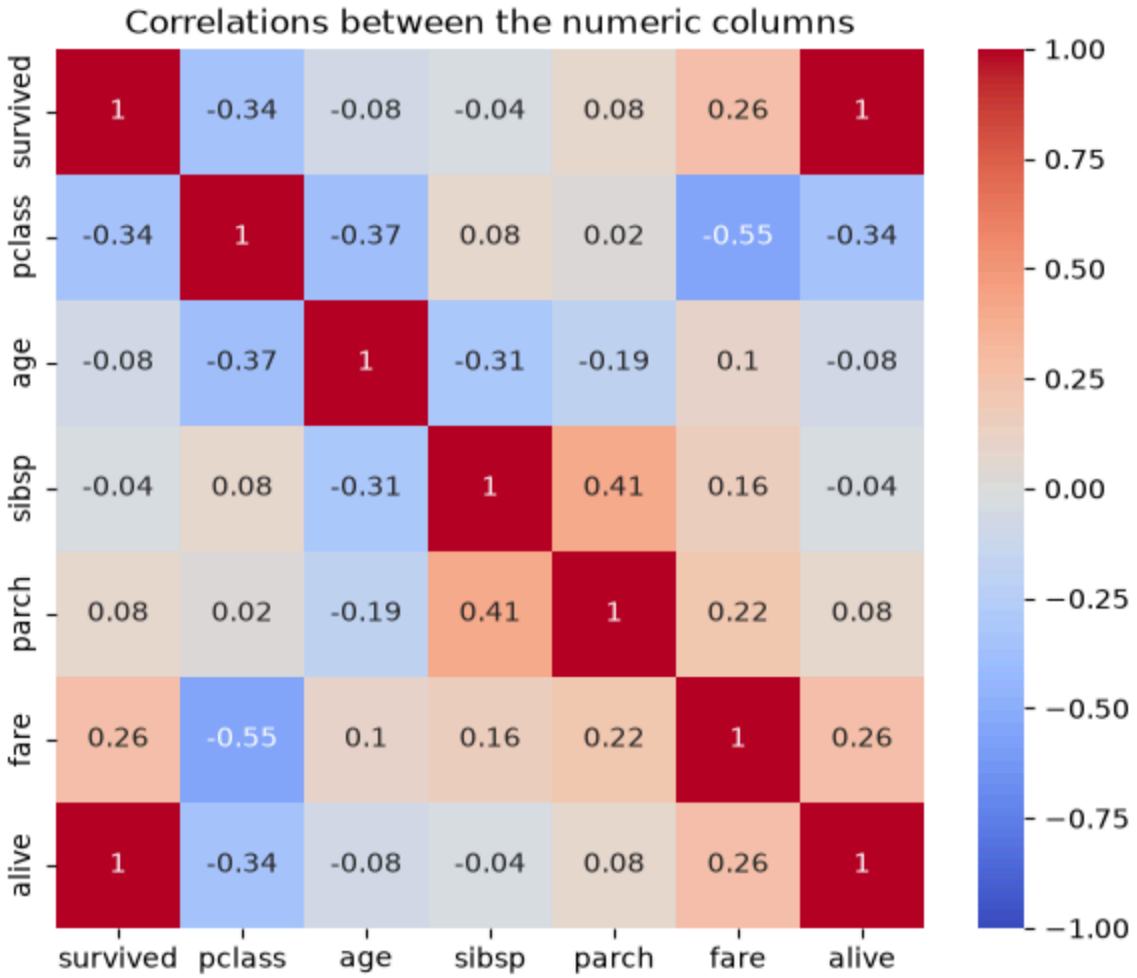
- (a) 15 features; 891 passengers have no missing value.
- (b) 14 features; 182 passengers have no missing value.
- (c) 14 features; the number of passengers with no missing value cannot be determined from this output.
- (d) 15 features; 203 passengers have no missing value.

> **Answer — (c).** 15 columns minus the target leaves 14 features. `info()` counts missing values column by column, so it cannot tell how many rows are complete; that needs `dropna()` (the true count is 182).

Question 17 — Short answer (4 points)

Exhibit 3

```
# Correlations between the numeric columns, with alive turned into 1 (yes) / 0 (no).
numeric_view = titanic[['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare']].copy()
numeric_view['alive'] = (titanic['alive'] == 'yes').astype(int)
plt.figure(figsize=(7, 6))
sns.heatmap(numeric_view.corr().round(2), annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlations between the numeric columns');
```



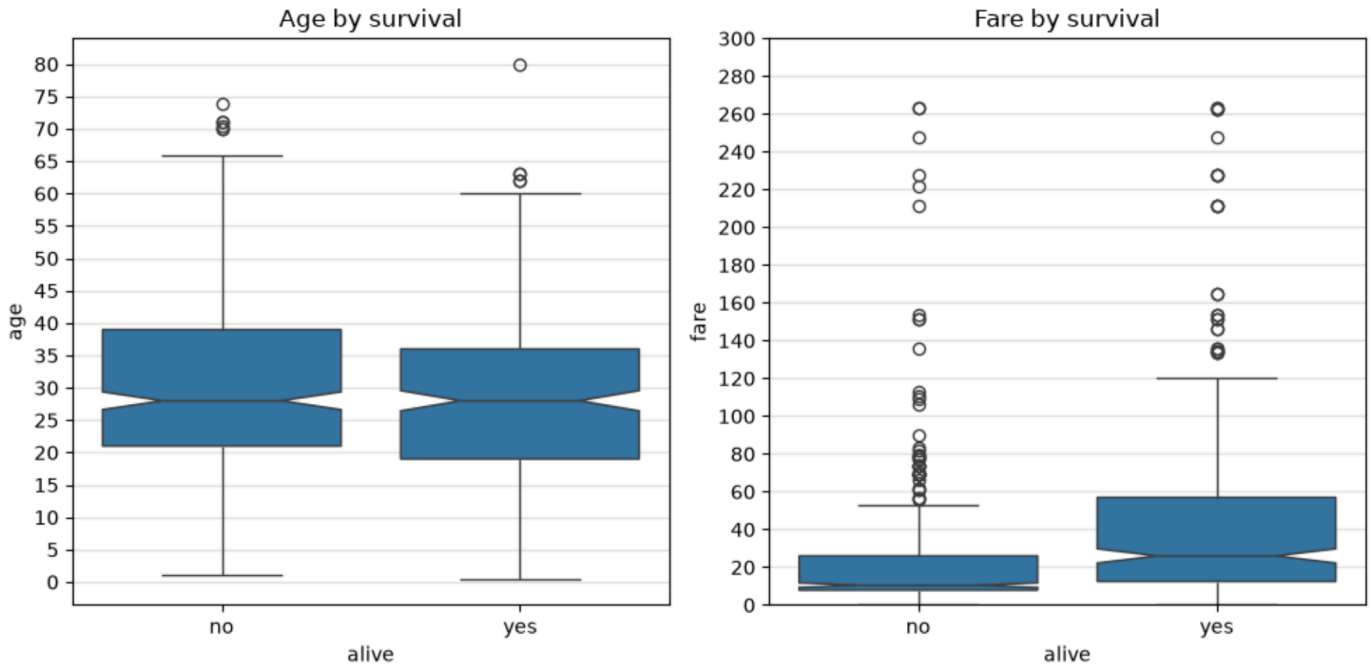
A colleague trains a model to predict `survived`, using all the other columns as features. What will the model learn? How well will it score on the test set, and would it be useful for predicting whether a passenger survives? Explain why. Answer in at most 4 sentences.

> **Model answer.** `alive` is `survived` in words (correlation 1), so the model just learns "alive = yes → survived". It scores about 100% even on the test set. It is useless for new passengers, because `alive` is not known in advance. This is leakage: drop `alive`.
>
> **Rubric (4 pts):** 1 pt each: `alive` duplicates the target · the model learns only that rule · it scores about 100% on test data · it is useless in practice (leakage).

Question 18 — Select all that apply (3 points)

Exhibit 4

```
# Notched boxplots of age and fare, for passengers who died and who survived.
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
sns.boxplot(data=titanic, x='alive', y='age', order=['no', 'yes'], notch=True, ax=axes[0])
sns.boxplot(data=titanic, x='alive', y='fare', order=['no', 'yes'], notch=True, ax=axes[1])
axes[0].set_yticks(range(0, 85, 5))
axes[1].set_yticks(range(0, 310, 20))
for ax in axes:
    ax.grid(axis='y', alpha=0.4)
axes[0].set_title('Age by survival')
axes[1].set_title('Fare by survival')
axes[1].set_ylim(0, 300)
plt.tight_layout();
```



Select **all** that are true.

- (a) The median fare of survivors is clearly higher than that of non-survivors.
- (b) The median age of survivors is clearly different from that of non-survivors.
- (c) Survivors' and non-survivors' median ages are about the same (around 28).
- (d) All passengers who survived paid a higher fare than all those who did not.

> **Answer — (a) and (c).** Notches that don't overlap mean the medians very likely differ. Fare: the notches are far apart (10.5 vs. 26). Age: they overlap and both medians are 28. (d) A higher median does not mean every survivor paid more.

Question 19 — Multiple choice (3 points)

Exhibit 5

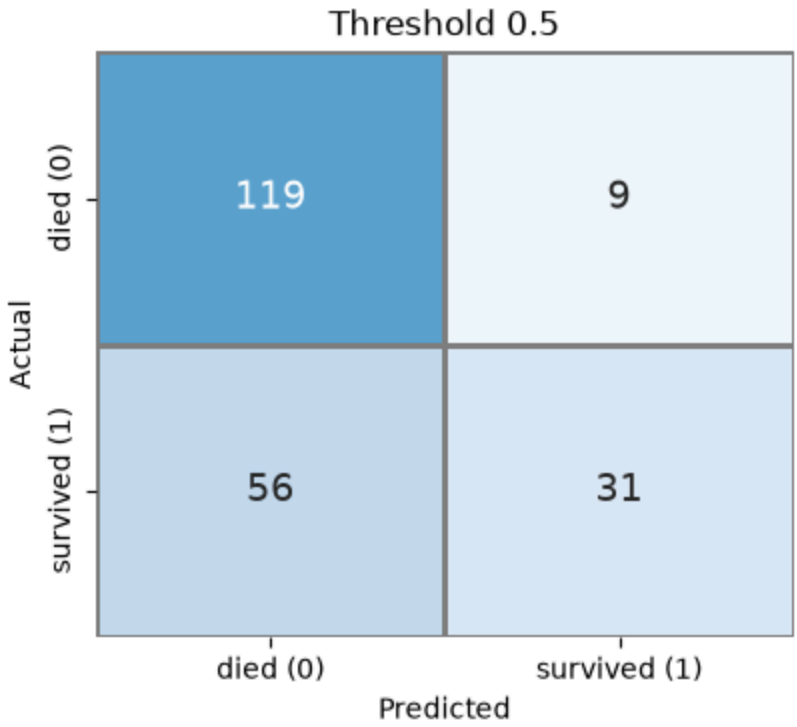
```
# Keep the numeric measurements only, and drop passengers with no age.
numeric = ['age', 'sibsp', 'parch', 'fare']
passengers = titanic[['survived'] + numeric].dropna()
X = passengers[numeric]
y = passengers['survived']

# Hold out 30% for testing and scale with the training part only.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)
```

Exhibit 6

```
# Logistic regression: probability of survival for each test passenger, cut at 0.5.
logit = LogisticRegression().fit(X_train_s, y_train)
prob = logit.predict_proba(X_test_s)[:, 1]
pred = np.where(prob >= 0.5, 1, 0)

# Draw the confusion matrix with labeled axes (plot_cm is a small plotting helper).
fig, ax = plt.subplots(figsize=(4.5, 3.8))
plot_cm(confusion_matrix(y_test, pred), 'Threshold 0.5', ax)
```



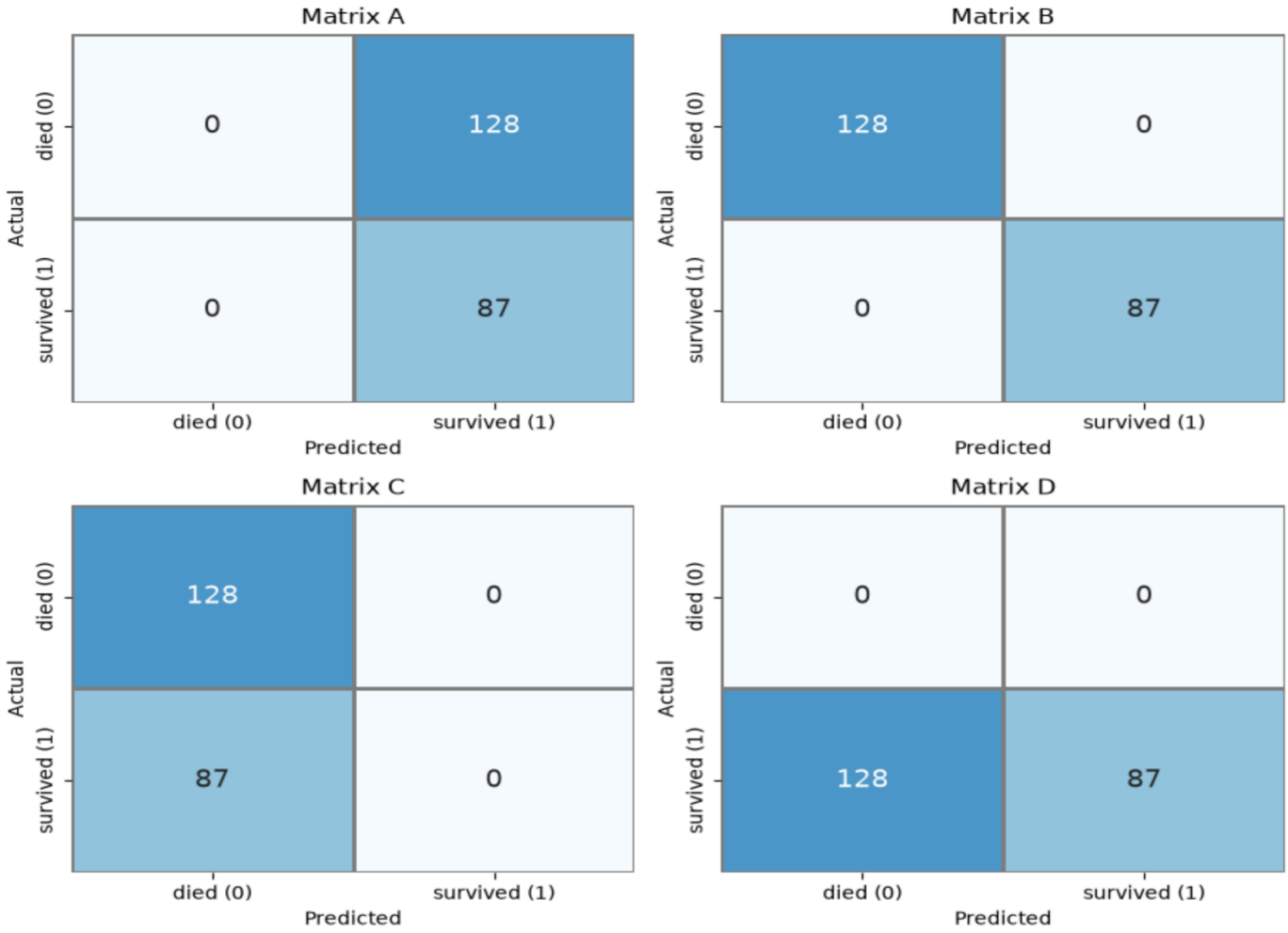
The threshold is now set to 1:

Exhibit 7

```
pred_1 = np.where(prob > 1, 1, 0)
```

Which of the following is the confusion matrix of `pred_1`?

Exhibit 8



- (a) Matrix A
- (b) Matrix B
- (c) Matrix C
- (d) Matrix D

> **Answer — (c) Matrix C.** No probability is above 1, so everyone is predicted 0. The row totals stay 128 died and 87 survived. A is threshold 0, B is a perfect model, and D has the axes swapped.

Case B2 — Breast Cancer (use Handout B2)

A hospital wants to flag **malignant** tumors from a biopsy image. For each of 569 tumors, software measured the cell nuclei in the image. Each column is the **mean** over the cells:

Column	Meaning
`mean radius`	Average distance from the center of a nucleus to its edge
`mean texture`	How much the gray level varies inside a nucleus
`mean perimeter`	Length of a nucleus's outline
`mean area`	Area of a nucleus

Column	Meaning
`mean smoothness`	How much the radius varies along the outline
`mean compactness`	Perimeter² / area − 1
`mean concavity`	How deep the inward dents in the outline are
`mean concave points`	How many inward dents the outline has
`mean symmetry`	How symmetric a nucleus is
`mean fractal dimension`	How irregular the outline is

Exhibit 1

```
# Load the breast cancer data and keep the first 10 columns (the mean of each measurement).
cancer = load_breast_cancer(as_frame=True)
X = cancer.data.iloc[:, :10]
y = 1 - cancer.target        # 1 = malignant, 0 = benign
print('Tumors:', X.shape[0], '  Features:', X.shape[1])
y.value_counts()
```

```
Tumors: 569   Features: 10
```

```
target
0    357
1    212
Name: count, dtype: int64
```

Exhibit 2

```
# Summary of each feature, one row per feature.
X.describe().T.round(3)
```

	count	mean	std	min	25%	50%	75%	max
mean radius	569.0	14.127	3.524	6.981	11.700	13.370	15.780	28.110
mean texture	569.0	19.290	4.301	9.710	16.170	18.840	21.800	39.280
mean perimeter	569.0	91.969	24.299	43.790	75.170	86.240	104.100	188.500
mean area	569.0	654.889	351.914	143.500	420.300	551.100	782.700	2501.000
mean smoothness	569.0	0.096	0.014	0.053	0.086	0.096	0.105	0.163
mean compactness	569.0	0.104	0.053	0.019	0.065	0.093	0.130	0.345
mean concavity	569.0	0.089	0.080	0.000	0.030	0.062	0.131	0.427
mean concave points	569.0	0.049	0.039	0.000	0.020	0.034	0.074	0.201
mean symmetry	569.0	0.181	0.027	0.106	0.162	0.179	0.196	0.304
mean fractal dimension	569.0	0.063	0.007	0.050	0.058	0.062	0.066	0.097

Question 20 — Multiple choice (3 points)

Exhibit 3

```
# Scale the features, fit PCA, and show every component.
X_scaled = StandardScaler().fit_transform(X)
pca = PCA().fit(X_scaled)
pd.DataFrame({
    'component': range(1, 11),
    'eigenvalue': pca.explained_variance_.round(2),
    'explained variance': pca.explained_variance_ratio_.round(3),
    'cumulative': pca.explained_variance_ratio_.cumsum().round(3),
})
```

	component	eigenvalue	explained variance	cumulative
0	1	5.49	0.548	0.548
1	2	2.52	0.252	0.800
2	3	0.88	0.088	0.888
3	4	0.50	0.050	0.938
4	5	0.37	0.037	0.975
5	6	0.12	0.012	0.987
6	7	0.08	0.008	0.995
7	8	0.03	0.003	0.999
8	9	0.01	0.001	1.000
9	10	0.00	0.000	1.000

How many components does the latent-root criterion keep, and how many are needed to keep at least 90% of the variance?

- (a) 2 and 3
- (b) 2 and 4
- (c) 3 and 4
- (d) 10 and 9

> **Answer — (b).** Eigenvalues above 1: only PC1 (5.49) and PC2 (2.52). The cumulative share passes 90% at 4 components (0.938).

Question 21 — Short answer (4 points)

Exhibit 4

```
# Hold out 30% of the tumors for testing, with the same class mix in both parts.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
print('Share malignant - training:', round(y_train.mean(), 3), '  test:', round(y_test.mean(), 3))
```

```
Share malignant - training: 0.372   test: 0.374
```

Exhibit 5

```
# k-NN with 5 neighbors on the raw measurements.
knn_raw = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
pred_raw = knn_raw.predict(X_test)
print('Accuracy:', round(accuracy_score(y_test, pred_raw), 3))
```

```
Accuracy: 0.889
```

In the **unscaled** model, which feature has the most influence on the prediction, and why? Answer in at most 2 sentences.

> **Model answer — `mean area`.** Unscaled k-NN distances are dominated by the feature with the largest spread, and area's SD (≈ 352) dwarfs every other feature's.
>
> **Rubric (4 pts):** 2 pts for `mean area` · 2 pts for the distance/spread reason (1 pt for just "it has the biggest numbers").

Question 22 — Multiple choice (3 points)

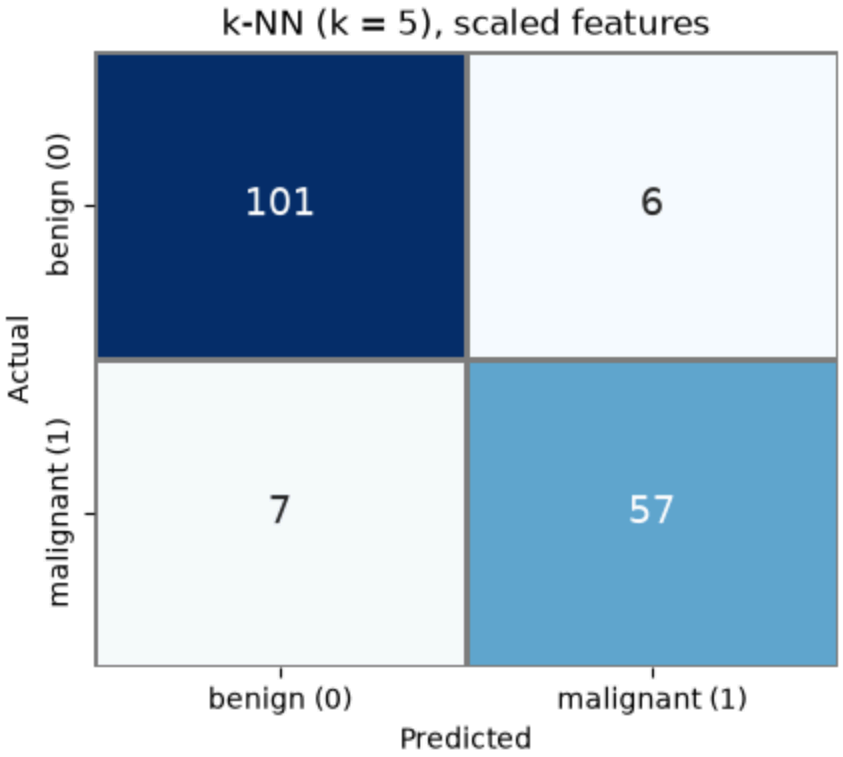
Exhibit 6

```
# Same model after standard scaling (the scaler is fit on the training part only).
scaler = StandardScaler().fit(X_train)
knn_scaled = KNeighborsClassifier(n_neighbors=5).fit(scaler.transform(X_train), y_train)
pred_scaled = knn_scaled.predict(scaler.transform(X_test))
print('Accuracy:', round(accuracy_score(y_test, pred_scaled), 3))
```

```
Accuracy: 0.924
```

Exhibit 7

```
# Confusion matrix of the scaled model, drawn with labeled axes.
fig, ax = plt.subplots(figsize=(4.8, 4))
sns.heatmap(confusion_matrix(y_test, pred_scaled), annot=True, fmt='d', cbar=False, cmap='Blues',
            linewidths=1, linecolor='gray', annot_kws={'size': 14},
            xticklabels=['benign (0)', 'malignant (1)'], yticklabels=['benign (0)', 'malignant (1)'],
ax=ax)
ax.set_xlabel('Predicted')
ax.set_ylabel('Actual')
ax.set_title('k-NN (k = 5), scaled features');
```



Is the model better at recognizing tumors that are really malignant, or tumors that are really benign?

- (a) Benign: it labels a larger share of the truly benign tumors correctly.
- (b) Malignant: it labels a larger share of the truly malignant tumors correctly.
- (c) Malignant: it gets 57 malignant tumors right and only 6 wrong.
- (d) Neither: it makes about the same number of mistakes on each kind (6 and 7).

> **Answer — (a).** Compare each row as a share of its total: benign 101/107 = 0.94, malignant 57/64 = 0.89. (c) and (d) compare counts, not shares.

Question 23 — True / False (2 points)

Look again at the `train_test_split` call in Exhibit 4. A colleague deletes `stratify=y` and reruns it.

True or false: without `stratify=y`, the test set will always have a lower share of malignant tumors than the training set.

- True
- False

> **Answer — False.** Without `stratify`, the split is random, so the test share can be lower, higher or equal. Only `stratify=y` guarantees equal shares.